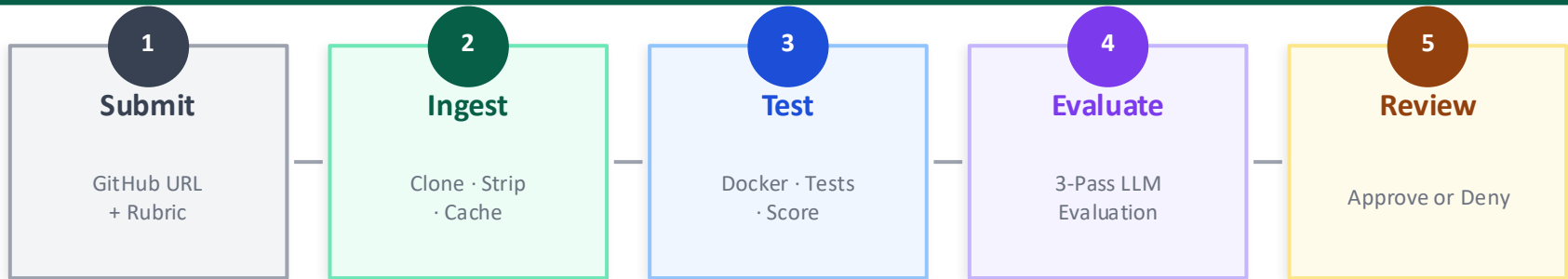


MAPLE A1

AI-Powered Code Submission Evaluator

Automated testing · Three-pass AI evaluation · Instructor review gates

How MAPLE A1 Works



Milestone 1 Core Infrastructure

Milestone 2 Sandboxed Execution

Milestone 3 AI Evaluation & RAG

MAPLE Standard Response Envelope

```
{ "success": bool, "data": { ... }, "error": null | string, "metadata": { ... } }
```

Every endpoint returns this same envelope — no surprises in the client.

SECTION 01

Infrastructure & DevOps

- DigitalOcean server · Nginx · TLS
- Docker sandbox security hardening
- pgvector schema + HNSW index
- GitHub Actions CI pipeline

Production Infrastructure

DigitalOcean Droplet

Ubuntu LTS · 4 GB RAM · 2 vCPU
161.35.125.120

Nginx + Let's Encrypt

TLS cert via Certbot
Reverse proxy → 127.0.0.1:8000

Systemd Service

maple-a1.service · Uvicorn
Auto-restart on failure

PostgreSQL 16 + pgvector

DigitalOcean Managed DB
3072-dim embeddings · HNSW index

GitHub Actions CI

Lint + tests on dev push
Manual deploy via SSH

Live Endpoints

API: <https://api.maple-a1.com>
App: <https://maple-a1.com>

Docker Security Sandbox

--no-new-privileges

Processes inside the container cannot escalate Linux capabilities

cap_drop: ALL

Every Linux capability dropped — minimum privilege principle

Read-only root FS

Filesystem is immutable; temp writes go to a dedicated tmpfs mount

30-second TTL

Containers auto-killed after 30 s; exit code 124 → timeout flag

CPU + Memory caps

Hard resource limits prevent denial-of-service from student code

exit 137 → OOM flag

Out-of-memory kills mapped to structured metadata for AI passes

Student code runs in an ephemeral sibling container via `/var/run/docker.sock` — it cannot reach the host filesystem.

SECTION 02

Backend & AI Pipeline

- PostgreSQL schema & ORM models
- Three-pass AI evaluation pipeline
- JSON schema validation + repair retry
- Deterministic scoring & NEEDS_HUMAN_REVIEW

Database & Core API

5 - TABLE SCHEMA

users

Instructor / Student role

assignments

Title, test repo URL, lint flag

rubrics

JSON grading schema

submissions

GitHub URL, commit hash, status

evaluation_results

AI feedback, scores, review status

KEY ENDPOINTS

POST /auth/login

OAuth2 Bearer JWT

POST /assignments

Create assignment

POST /evaluate

Trigger evaluation → submission_id

GET /submissions/{id}

Poll status + AI feedback

POST /submissions/{id}/review

Instructor approve / reject

LLM Redactor — strips GitHub PATs, email addresses, and env var values before any external API call

Three-Pass AI Evaluation Pipeline

PASS
1

Test Reconciliation

Gemini 3.1 Pro • 60s timeout

Runs unconditionally on every submission

→ Logic bug

→ Environment / dependency issue

→ Timeout or OOM

Classifies each failing test

PASS
2

Style Review

Gemini 3.1 Flash Lite • 30s timeout

Conditional — fires only when:

→ Rubric has style criteria, OR

→ Linter found violations

RAG: top-5 pgvector chunks

(cosine $\geq$ 0.75 threshold)

PASS
3

Synthesis

Gemini 3.1 Pro • 60s timeout

Combines Pass 1 + 2 output into:

→ Per-criterion level scores

Exemplary / Proficient / ...

→ Confidence value per criterion

→ Code recommendations w/ line #

Schema Validation & Scoring

LLM OUTPUT VALIDATION

LLM Response

JSON Schema Validation

Pass → Persist Result

Fail → 1 Repair Retry

Still Fail → EVALUATION_FAILED

DETERMINISTIC SCORING

Test Pass Ratio

$\text{passed} \div (\text{passed} + \text{failed} + \text{errors})$
Mapped to rubric criterion weights

0–100 Deterministic Score

Baseline independent of AI
Stored alongside ai_feedback_json

NEEDS_HUMAN_REVIEW

Triggered by: ambiguous rubric, confidence < 0.5,
or RAG no-match (threshold 0.75)

SECTION 03

Ingestion, Cache, & Angular

- GitHub repo cloning & preprocessing
- SHA + rubric digest caching (fcntl)
- Angular 21 submission form
- Status polling & instructor review UI

GitHub Ingestion & Caching

INGESTION PIPELINE

Validate GitHub URL

Regex: `github.com/{owner}/{repo}`

Clone with PAT

Private repos via GitHub Personal Access Token

Preprocessing

Strip: `.git` · `node_modules` · `venv` · `binaries`

Cleaned repo ready

CACHE KEY DESIGN

Commit Hash

`git HEAD`

+

Rubric Digest

SHA-256 of JSON

16-char SHA-256 path token

Unique key per commit × rubric pair

fcntl file locking — prevents race conditions
on concurrent submissions from the same repo

Same commit × same rubric → instant cache hit. Either changes → full re-evaluation.

Angular Frontend

SUBMIT PAGE

GitHub Repository URL

`https://github.com/student/my-repo`

Assignment ID

`assignment-001`

Rubric File (.json)

`rubric.json`

Submit for Evaluation

STATUS PAGE

Reactive Polling

GET /submissions/{id} every 3 s
via setInterval → RxJS subscribe

Terminal Statuses

Completed · Failed
Awaiting Review · EVALUATION_FAILED

Angular 21 Stack

Standalone components · ReactiveFormsModule
TypeScript 5.9 · RxJS · HttpClient

Instructor Review UI

CRITERIA SCORES COMPONENT

Test Coverage

Exemplary

92

91% confidence

Code Structure

Proficient

78

72% confidence

Style Guide

Needs Review

—

Error Handling

Developing

55

44% confidence

DIFF VIEWER + REVIEW PANEL

```
src/utils/validator.py : lines 14-18
- - def validate(data):
+ + def validate(data: dict) -> bool:
    if not data:
+ +     raise ValueError('Empty input')
    return schema.validate(data)
```

Instructor Review Panel

AI feedback is hidden until instructor approves.
Approval triggers visibility release to the student.

Approve

Reject + Notes

MAPLE A1

Milestone Status

Milestone 1

Core Infrastructure

Complete

Milestone 2

Sandboxed Execution

Complete

Milestone 3

AI Evaluation

LLM stub pending

What's Next

- Jayden completes `llm.complete()` → retry + fallback chain activates the full M3 pipeline
- Style guide ingestion populates pgvector for live RAG retrieval in Pass 2
- End-to-end demo: student submits → Docker tests → AI evaluates → instructor reviews → result released

Thank you — <3